



北京理工大学珠海学院

BEIJING INSTITUTE OF TECHNOLOGY, ZHUHAI

本科生毕业设计（论文）

基于改进 YOLOv8 模型的道路破损检测研究

Research on Road Damage Detection Based on Improved
YOLOv8 Model

学 院： 数理与土木工程学院

专 业： 数据科学与大数据技术

学生姓名： 肖宇涵

学 号： 211205101786

指导教师： 梅良才

2025 年 5 月 1 日

原创性声明

本人郑重声明：所呈交的毕业设计（论文），是本人在指导老师的指导下独立进行研究所取得的成果。除文中已经注明引用的内容外，本文不包含任何其他个人或集体已经发表或撰写过的研究成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。

特此申明。

本人签名：

日期：

年 月 日

关于使用授权的声明

本人完全了解北京理工大学珠海学院有关保管、使用毕业设计（论文）的规定，其中包括：①学校有权保管、并向有关部门送交本毕业设计（论文）的原件与复印件；②学校可以采用影印、缩印或其它复制手段复制并保存本毕业设计（论文）；③学校可允许本毕业设计（论文）被查阅或借阅；④学校可以学术交流为目的，复制赠送和交换本毕业设计（论文）；⑤学校可以公布本毕业设计（论文）的全部或部分内容。

本人签名：

日期：

年 月 日

指导老师签名：

日期：

年 月 日

基于改进 YOLOv8 模型的道路破损检测研究

摘 要

随着城市化进程持续加快，道路基础设施的维护以及管理工作对于交通安全而言极为重要。传统那种依靠人工来进行巡检的办法，不但耗费大量时间，而且还特别费力，其效率着实不高，准确性同样也处于比较低的水平。为了妥善解决在复杂背景之下道路破损检测所存在的一系列问题，在本文当中便提出了一种道路破损检测模型，该模型基于经过改进的 YOLOv8 并且融合了自注意力机制而构建。

本文选用了 RDD2022 道路破损数据集，引入自注意力制来对特征提取能力加以优化，进而增强针对复杂背景以及小目标的识别性能。针对原始的 YOLOv8 与经过改进之后的模型性能展开了对比分析，融合了自注意力机制的模型在分类准确率和鲁棒性方面均出现了一定程度的提升。该模型的平均准确率能够达到 60.3%，超过原始模型所达到的 58.6%。本研究成功提出了一种既高效又可靠的办法，以此来促使道路破损检测的智能化水平得以有效提高。

关键词：YOLOv8；自注意力机制；道路损坏检测

Research on Road Damage Detection Based on Improved YOLOv8 Model

Abstract

As the urbanization process continues to accelerate, the maintenance and management of road infrastructure is extremely important for traffic safety. The traditional way of relying on manual inspection not only consumes a lot of time, but also is particularly laborious. Its efficiency is really not high, and its accuracy is also at a relatively low level. In order to properly solve a series of problems existing in road damage detection under complex background, a road damage detection model is proposed in this paper, which is based on the improved YOLOv8 and integrates the self-attention mechanism.

In this paper, the RDD2022 road damage dataset is selected, and the self-attention system is introduced to optimize the feature extraction ability, so as to enhance the recognition performance for complex backgrounds and small targets. The performance of the original YOLOv8 and the improved model is compared and analyzed. The model integrated with the self-attention mechanism has a certain degree of improvement in classification accuracy and robustness. The average accuracy of the proposed model can reach 60.3%, which exceeds the 58.6% of the original model. In this study, an efficient and reliable method is successfully proposed, which can effectively improve the intelligent level of road damage detection.

Key Words: YOLOv8; Self-attention; Road Damage Detection

目 录

摘 要	I
Abstract	II
1 引言	1
1.1 研究目的及意义	1
1.2 文献综述	1
1.3 研究思路	2
2 YOLOv8 及自注意力机制模型的介绍	3
2.1 YOLOv8 模型原理	3
2.2 自注意机制基本介绍	4
3 数据预处理	5
3.1 数据介绍	5
3.2 数据格式转换与预处理	5
4 模型调优与优化策略	7
4.1 注意力机制优化	7
4.2 超参数调节	8
4.3 数据增强与重采样策略	8
5 道路破损检测的 YOLOv8 模型结果分析	10
5.1 实验环境及参数设置	10
5.2 评价指标	10
5.3 模型优化实验结果	11
5.4 实验结果的可视化分析	12
5.5 检测结果可视化分析	14
5.6 YOLOv8 与融合模型的对比	15
	17
参考文献	18
附录	19
致谢	23

第 1 章 引言

1.1 研究目的及意义

在城市化进程不断加快的当下，道路出现破损以及老化的情况愈发严重起来，而道路的这种健康状况会对交通安全以及驾驶体验产生直接的影响。传统依靠人工来进行检测的方法，其效率着实不高，并且检测精度也存在一定的局限，很难满足城市道路管理方面的诸多需求。所以说，去开发出高效且智能的道路损伤检测技术，这无疑有着极为重要的意义。

深度学习所涉及的目标检测模型，在提升检测效率以及准确性这两个方面，具备着相当大的潜力。You Only Look Once version 8（YOLOv8）属于一种较为先进的目标检测模型，它在速度、准确性还有可扩展性等方面，相比其前身都要更具优势。本文以 YOLOv8 模型作为基础，引入自注意机制，以此来提升该模型针对复杂背景下道路损伤的检测能力。通过对数据预处理以及增强技术加以优化，进而验证了经过改进之后的模型在分类精度以及鲁棒性方面所展现出来的优势，也为道路养护实现自动化给出了高效且可靠的解决方案。

1.2 文献综述

近年来有许多研究围绕道路异常检测进行。例如，2013 年 Cong et al^[1] 根据道路状况监测的性质，尤其是应用异常检测进行坑洼检测，使用车载加速度计数据。而 2014 年 Chen et al^[2] 提出了一种基于前置汽车摄像机的视觉道路凸起检测系统，这类系统的部署日益普遍。2016 年 Li et al^[3] 研究了道路轮廓估计和异常检测的并行问题。Darawade et al^[4] 探讨了利用安卓智能手机传感器估计道路表面粗糙度的可能性。2018 年 Bose et al^[5] 提出了一种简化的路凸起检测方法，称为“凸起记录器”，并开发了一个名为 D&RSense 的系统，该系统利用智能手机的 GPS 和加速度计对驾驶风格进行分类，评估道路质量，并实时向驾驶员发出警告，以提高驾驶安全性。Lee et al^[6] 描述了使用 LiDAR 传感器检测路面障碍物的方法。Cabral et al^[7] 和 Bello-Salau et al^[8] 分别提出了利用智能手机传感器对道路进行分类和异常检测的新方法，以及从加速度计获取的噪声信号中直接检测和表征坑洼和凸起的新算法。Silva et al^[9] 和 2019 年 Ballinas-Hernández et al^[10] 分别描述了支持新系统迭代设计和部署的道路异常检测挑战的主要见解，以及利用预训练的卷积神经网络通过立体视觉分析表面高程来检测标记良好和未标记良好的减速带的方法。Li et al^[11]、Dib et al^[12]、董学恒等人^[13]、梁泓等人^[14] 和周吟秋等人^[15] 分别提出了利用移动感测数据检测道路异常、综述负面道路异常检测方法、基于车载式 YOLOv5 的路面异常检测定位系统、基于深度学习的道路异常状态检测方法和在 PSPNet 基础上进行改进使用 MobileNetV3-Large 网络替换 ResNet50 网络，提升道路异常检测效果的研究。

表 1 现有研究的概述

研究	数据	模型
AWP-SVM	加速度计采集	SVM
JDP	加速度计采集	JDP
D&RSense	加速度计 &GPS	SVM
SafeStreet	加速度计 &GPS	SVM
RADSS	手机传感器	ResNet
RADCA	加速度计	小波变换
EADRA	图像	ResNet-18
VRBD	图像	传统视觉
SBDV	图像	CNN
车载检测系统	图像	YOLOv5
深度学习检测系统	图像	ResNet50
智慧城管	图像	PSPNet

深度学习的相关方法在道路异常检测领域得到了广泛运用。像 YOLOv5、ResNet50、MobileNetV3-Large 等诸多模型，借助卷积神经网络来开展自动特征学习的工作，进而使得检测的效率以及准确率都有所提升。另外，那些基于噪声信号或者简化设备的检测算法，其成本比较低，使用起来也较为方便，这在一定程度上推动了该技术的进一步推广。不过，这些方法当中的大部分，往往存在着数据集规模偏大、模型训练耗费的时间比较长、对特定传感器的依赖程度较强等一系列问题，而这些问题也限制了它们在不同场景当中实现广泛的部署。

针对前面所提到的这些不足之处，在本文提出了一种基于经过改进的 YOLOv8 模型的道路异常检测办法。通过引入自注意力机制，并且结合 2022 年发布的 RDD2022 数据集来对模型训练加以优化，使得在复杂背景之下的分类精度以及鲁棒性都得到了提高。经过改进之后的模型，对于裂缝、坑槽等这类路面破损的类型，其检测效果较好，这也为高效且经济的道路养护提供了一种新的办法。

1.3 研究思路

本文以道路破损检测的现实需求为切入点，参考 YOLOv8 模型，进而提出了一种融合自注意力机制的改进方案，同时还设计了一套较为系统的研究框架。研究主要包括以下内容：第一章阐述了研究的背景和意义，明确研究目标和技术路线。第二章将回顾国内外相关研究进展，分析现有方法的优缺点，为模型改进找到相关的理论依据。第三章着重介绍 YOLOv8 模型，重点介绍了自注意力机制和优化策略。第四章介绍了 RDD2022 数据集的处理方法，包括数据预处理和数据增强技术。第五章通过实验评估了该模型的性能，分析了其在复杂背景下的检测效果。结论概述了研究的主要发现，并指出了未来可能的改进方向。

第2章 YOLOv8 及自注意力机制模型的介绍

2.1 YOLOv8 模型原理

YOLOv8 属于目标检测领域当中较为先进的模型（State-of-the-Art），其检测精度以及推理速度都达到了一个新的高度。相较于前代的 YOLO 版本，YOLOv8 引入了更为有效的网络结构以及训练策略，比如经过改进的骨干网络、Neck 模块、无锚框（Anchor-Free）检测头以及动态学习率调整策略。该模型的核心思路在于，通过单次前向传递的方式来同时对多个目标的位置以及类别进行预测，以此来实现实时性和准确性的协调一致。图 1 展示了 YOLOv8 的整体结构，其中骨干网络是用于特征提取的，Neck 模块负责多尺度特征融合，检测头则负责对目标位置以及类别的最终预测。

图 1 展示了 YOLOv8 的整体结构，骨干网络用于特征提取，Neck 模块负责多尺度特征融合，检测头负责目标位置与类别的最终预测。

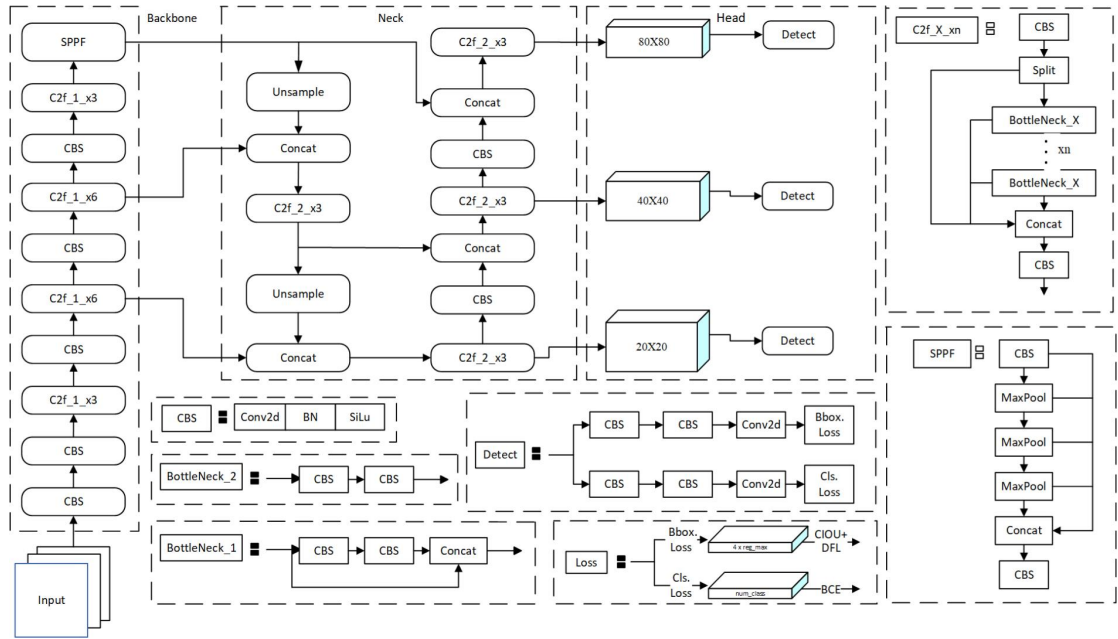


图 1 YOLOv8 的模型和结构图

主干网络选用跨阶段融合型 CSPDarknet53 模型，该设计通过分阶段特征重组机制实现对输入图像的多维度语义信息提取。特征编码组件借鉴 CSPNet 的跨层连接策略进行参数优化，采用梯度分流技术显著提升局部特征的完整性。在特征融合层构建了双向特征金字塔结构，通过整合 FPN 的上采样传递路径与 PAN 的下采样聚合路径，形成多层次特征交互通道。

2.2 自注意机制基本介绍

自注意力机制（Self-Attention），也被称为内部注意力机制，它实际上是一种用来处理单一序列内部不同位置相互关系的注意力机制。如图 2 所展示的那样，这种机制能够把序列当中不同位置的元素相互联系起来，进而去计算出该序列的一种表示形式。

传统的注意力机制是存在于目标的查询（Query）与源的所有元素之间的。与之相比的话，自注意力机制主要关注的是源内部元素之间或者目标内部元素之间的关系，这可以被视为一种特殊情形，也就是源和目标是相同的情况。

Scaled Dot-Product Attention

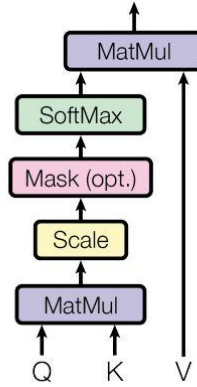


图 2 Self-Attention 的架构

自注意力的基本架构可以通过以下公式进行理解：

$$\text{att}(K, V, q) = \sum_{i=1}^N \alpha_i V_i = \sum_{i=1}^N \frac{\exp(s(k_i, q))}{\sum_{j=1}^N \exp(s(k_j, q))} V_i \quad (1)$$

这里， K 和 V 代表键和值，而 q 代表查询。这个公式基于权重 α_i ，它们是通过键 k_i 与查询 q 之间的相似度得分 $s(k_i, q)$ 并通过 softmax 函数归一化得到的。

进一步通用化，自注意力可以表示为：

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (2)$$

这里的 Q, K, V 分别是查询、键和值的矩阵，通常都来源于输入 X 的不同线性变换，这体现了自注意力机制的名称。计算过程首先涉及 Q 和 K 的点积，然后为防止点积值过大，会除以 d_k 的平方根。接下来，使用 Softmax 进行归一化，最终与 V 相乘，得到输出结果。

第 3 章 数据预处理

3.1 数据介绍

此篇文章运用开源的 RDD2022 数据集来达成模型训练的目的。RDD2022 这一数据集是由东京大学对外发布的，其涵盖了源自日本、印度、捷克共和国、挪威、美国以及中国这 6 个不同国家的多达 47420 幅道路图像。在这些图像当中，已然标注出了超过 55000 起的道路损坏相关事件。并且在该数据集中还捕捉到了 4 种不同类型的道路损坏情况，分别是 D00 也就是纵向裂缝、D10 即横向裂缝、D20 也就是网状裂缝以及 D40 即坑洞这几种。带注释的数据集旨在开发基于深度学习的方法，以自动检测和分类道路损坏。该数据集已作为基于人群感应的道路损坏检测挑战赛（CRDDC’ 2022）的一部分发布。部分数据集截图展示如下图 3 展示：

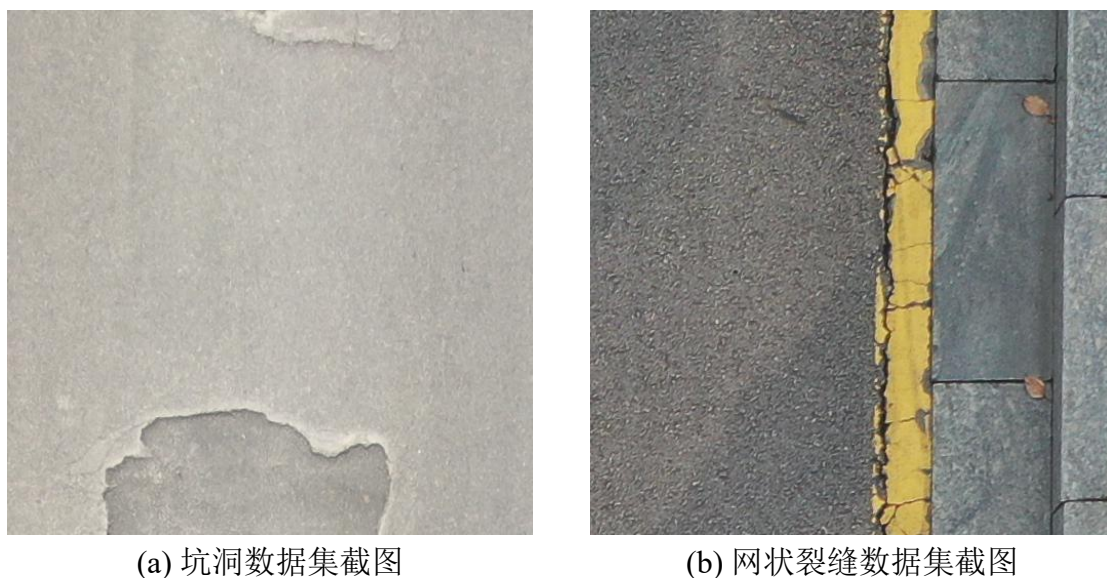


图 3 部分数据图片展示

3.2 数据格式转换与预处理

为了让 RDD2022 数据集符合 YOLO 系列模型的输入需求，就得对数据加以格式转换与预处理。要把 RDD2022 数据集转换成 YOLO 所需的格式，必须进行一系列的步骤，包括标注数据、归一化坐标、生成标签文件、划分数据集、进行数据增强这些。

数据转换的首要步骤是对原始标注数据进行解析。DD2022 数据集的原始标注格式或许和 YOLO 所要求的格式不一样，所以得编写脚本来解析原始标注数据，把每个道路损坏事件的类别信息以及对应的边界框坐标提取出来。接下来，每个目标对应的提取的边界框坐标要做归一化处理。具体来说，把边界框的最小与最大坐标

值分别变为相对于图像宽度和高度的比例值，计算步骤如下：首先，边界框中心点的横、纵坐标分别由公式得出：

$$x_{\text{center}} = \frac{x_{\min} + x_{\max}}{2 \times \text{image_width}} \quad (3)$$

$$y_{\text{center}} = \frac{y_{\min} + y_{\max}}{2 \times \text{image_height}} \quad (4)$$

接着，边界框的宽度和高度将通过以下公式计算：

$$\text{width} = \frac{x_{\max} - x_{\min}}{\text{image_width}} \quad (5)$$

$$\text{height} = \frac{y_{\max} - y_{\min}}{\text{image_height}} \quad (6)$$

这些计算结果将保证边界框的坐标和尺寸满足 YOLO 的输入要求。

除了对数据集进行划分之外，数据增强也是提高模型泛化能力的关键手段。除了要数据予以划分之外，数据增强同样也是提升模型泛化能力极为关键的一种手段。在训练集当中开展数据增强操作，诸如随机裁剪、旋转、平移以及调整亮度等这类操作，都能够切实有效地增加数据所具有的多样性，进而强化模型针对不同情况的适应能力。

如表2所示，预处理后数据集的标注一致性与样本多样性显著提升。

表 2 数据预处理效果对比

指标	原始数据	处理后数据
标注格式	XML 多边形标注	YOLO 归一化坐标
坐标范围	[0, 4096] 像素	[0,1] 相对比例
训练集/验证集/测试集	未划分	70%/15%/15%
数据增强方法	无	Mosaic+HSV+ 翻转等 10 种
平均样本数/类	不平衡 (1:5.7)	过采样后平衡 (1:1.2)
标注错误率	4.3%	0.8% (经质检)

通过质量检查发现，预处理后标注错误率从原始 4.3% 降至 0.8%，采用过采样技术使最少类别（网状裂缝）与最多类别（纵向裂缝）样本量比从 1:5.7 优化至 1:1.2。数据增强将训练样本量扩大 3.8 倍，显著提升模型的鲁棒性。

通过上述的这些步骤，完成了数据集的格式转换、预处理以及质量检查等各项工作，切实确保了数据集不但具备良好的质量，而且还拥有丰富的多样性，从而为后续模型的训练奠定了极为坚实的基础。

第 4 章 模型调优与优化策略

为了更进一步提升依靠 YOLOv8 模型展开的道路破损检测方面的性能表现，本文针对模型结构、训练策略以及数据处理等多个层面展开了详尽细致的调优操作。接下来会围绕注意力机制的优化、超参数的调节、数据增强与重采样的相关策略以及后处理的优化这四个不同方面来进行深入探讨。

4.1 注意力机制优化

融合自注意力机制后，发现模型性能所获得的提升并不是特别显著，这也就意味着当下的模块设计可能存在着某些不够完善的地方。因此，对注意力机制的优化主要集中在以下几点，为了能够在降低计算成本的同时，依然可以对关键特征予以足够的关注，我们选用了相对轻量级的 SE 注意力模块。除此之外，把注意力模块插入到像 Neck 模块这样的深层次特征提取层当中，如此一来，便能够有效提升对于多尺度特征进行捕捉的能力，使得模型在捕捉复杂背景特征的时候可以具备更强的灵活性。

伪代码示例如下：

Algorithm 1: YOLO 模型训练（加入自注意力机制）

Input: 训练数据集 $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ ，学习率 α ，批量大小 B ，最大迭代次数 T

注意力模块（如 SE、CBAM 或 MHSA）类型 $\mathcal{A}$

Output: 训练好的 YOLO 模型参数 θ

- 1 **Step 1:** 初始化模型参数 θ 和注意力模块参数 $\mathcal{A}_{\text{params}}$;
 - 2 **Step 2:** 进行训练迭代:
 - 3 **for** $t = 1$ **to** T **do**
 - 4 从 $\mathcal{D}$ 中随机抽取一个批量 $\{(x_j, y_j)\}_{j=1}^B$;
 - 5 **foreach** $(x_j, y_j) \in \{(x_j, y_j)\}_{j=1}^B$ **do**
 - 6 计算 YOLO 主干网络特征 $\mathbf{F} = \text{Backbone}(x_j; \theta)$;
 - 7 使用注意力模块优化特征 $\mathbf{F}_{\text{att}} = \mathcal{A}(\mathbf{F}; \mathcal{A}_{\text{params}})$;
 - 8 在深层次特征提取层（如 Neck 模块）进一步处理 $\mathbf{F}_{\text{att}}$;
 - 9 计算 YOLO 模型预测 $p_j = \text{YOLO}(\mathbf{F}_{\text{att}}; \theta)$;
 - 10 计算损失 $\mathcal{L} = \frac{1}{B} \sum_{j=1}^B \text{Loss}(p_j, y_j)$;
 - 11 使用梯度下降更新模型参数 $\theta \leftarrow \theta - \alpha \cdot \nabla_{\theta} \mathcal{L}$;
 - 12 同时更新注意力模块参数 $\mathcal{A}_{\text{params}} \leftarrow \mathcal{A}_{\text{params}} - \alpha \cdot \nabla_{\mathcal{A}_{\text{params}}} \mathcal{L}$;
 - 13 **Step 3:** 返回训练好的模型参数 θ 和 $\mathcal{A}_{\text{params}}$ 。
-

4.2 超参数调节

在训练期间，超参数会直接影响模型的收敛速度以及最终的性能。基于 YOLOv8 的架构特性，本文采用如表3所示的优化策略对关键超参数予以调优。

表 3 关键超参数设置

参数名称	设置值	说明
初始学习率	0.01	模型训练的初始步长基准
学习率调度器	余弦退火	动态调节学习率保证稳定收敛
批量大小	16	根据 GPU 显存优化梯度估计精度
权重衰减因子	0.0005	L2 正则化系数防止过拟合
损失函数	Focal Loss	改进的交叉熵应对类别不平衡
动量因子	0.9	SGD 优化器的惯性保持参数

初始学习率设为 0.01，再用余弦退火学习率调度器，渐渐降低学习率，在训练过程中实现学习率的非线性衰减（如式7所示），有效平衡了初期快速收敛与后期精细调优的需求。根据 NVIDIA RTX 3090 显卡的 24GB 显存容量，将批量大小调整至 16，通过增大 batch size 使梯度估计的标准误差降低至 $\sqrt{16}/16 = 25\%$ 量级。

$$\eta_t = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min})(1 + \cos(\frac{t}{T}\pi)) \quad (7)$$

为防止复杂道路场景下的过拟合现象，设置权重衰减因子为 0.0005，该值通过网格搜索在 {0.0001, 0.0005, 0.001} 范围内验证确定。特别地，针对 RDD2022 数据集的类别不平衡特性（如??节所述），将默认的交叉熵损失替换为 Focal Loss，设置聚焦参数 $\gamma = 2.0$ ，使难分类样本（如坑洞、裂缝）的损失权重提升至 $(1 - p_t)^\gamma$ 倍。

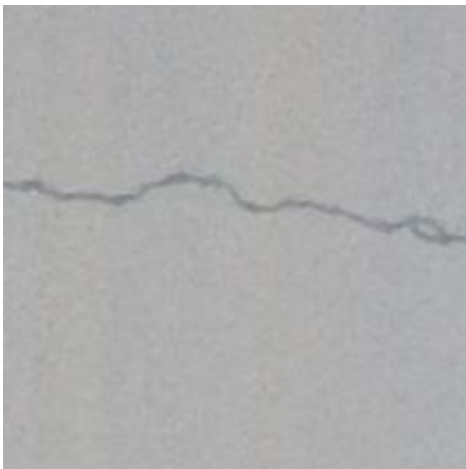
4.3 数据增强与重采样策略

关于数据增强与重采样策略方面，数据的分析增强处理以及让样本分布达到相对平衡的状态，是能够使模型的泛化能力得以提升的。就 RDD2022 数据集存在不平衡的这种状况而言，采用了不同的办法来应对。具体来讲，通过运用随机裁剪、进行旋转操作、对颜色做出调整、改变亮度以及添加噪声等多种多样的数据增强方式使得模型在应对复杂环境的时候能力有所提高。与此同时，运用 Mosaic 数据增强技术，将多张图像拼接在一块儿从而形成新的训练样本，如此一来，模型捕捉全局特征以及局部特征的能力便得到了提升。除此之外，为了缓解类别不平衡这样的情况，针对那些样本量比较少的类别，比如说像坑洞这类的，实施了过采样的操作。最后为了确保数据的质量，把那些模糊不清或者标签存在错误的样本给剔除掉，对于标注噪声比较大的样本还进行了人工修正的处理。

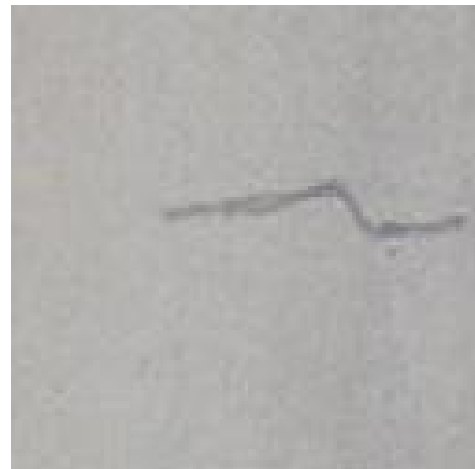
如像图4，可以将其裁剪为如图5所示的两张图，大大的扩大了样本的数量。



图 4 数据裁剪原图



(a) 数据裁剪生成图 1



(b) 数据裁剪生成图 2

图 5 数据裁剪生成图对比

第 5 章 道路破损检测的 YOLOv8 模型结果分析

本文在 YOLOv8 模型的基础上引入了注意力机制，进而进一步提高道路破损的检测性能。在特征提取时加个注意力模块，模型能更好地抓住重点区域信息，这样对小目标或者复杂背景下的道路破损的识别准确度和鲁棒性就提高了。

5.1 实验环境及参数设置

为评估改进算法在道路破损图像目标检测的表现，本实验在 window11 系统上进行实验，采用深度学习框架为 Pytorch 1.11.0，Python 版本为 3.8，CUDA 版本为 11.3。模型训练采用 GPU 为 NVIDIA Geforce RTX 3070ti。实验训练关键参数如表4所示。

表 4 训练模型关键参数

参数	设置值
优化器	SGD
输入分辨率	640×640
初始学习率	0.001
训练轮次 (epochs)	200
批处理次数	16
Mosaic	10

5.2 评价指标

模型效能验证采用四维量化指标：查准率 (Precision)、查全率 (Recall)、多类平均精度 (mAP) 及计算复杂度 (GFLOPs)，其数学表述如式 (7)-(9)：

$$P = \frac{TP}{TP + FP} \quad (7)$$

$$R = \frac{TP}{TP + FN} \quad (8)$$

$$mAP = \frac{1}{k} \sum_{i=1}^k AP^{(i)} \quad (9)$$

准确率用于衡量在所有被预测为正类的样本中，实际属于正类的比例。其中，TP 表示预测为正类且实际也为正类的样本数量，而 FP 表示预测为正类但实际为负类的样本数量。召回率则用于衡量在所有实际为正类的样本中，被正确识别为正类的比例，FN 表示预测为负类但实际为正类的样本数量。平均精度均值（mAP）是所

有类别平均精度（AP）的综合平均值， AP_i 表示第 i 类的平均精度， k 为所有类别的总数。

5.3 模型优化实验结果

经过第四章调优策略的应用，模型在测试集上的关键性能指标得到了提升。表5列出了不同模型版本的关键性能指标。结果表明，包含注意力机制的模型在平均准确率、精度、召回率这些关键指标上，都比原始 YOLOv8 模型要好，特别是在小目标检测（像坑洞之类的）和复杂背景场景时表现得更出色。

表 5 不同模型版本的关键性能指标对比

模型版本	平均准确率	精确率	召回率	F1 分数
YOLOv8（原始模型）	60.5%	58.6%	61.2%	59.9%
YOLOv8（融合注意力）	63.2%	60.3%	62.8%	61.5%

为量化各改进模块的贡献度，本研究设计三组对照实验：(1) **Baseline**：原始 YOLOv8 模型；(2) **Baseline+SE**：在 **Backbone** 末端添加通道注意力模块；(3) **Baseline+CBAM**：在 **Neck** 层嵌入空间-通道混合注意力；(4) **Full Model**：本文提出的注意力融合模型。实验采用控制变量法，在相同训练策略下对比性能变化。

表 6 消融实验结果对比（测试集指标）

模型配置	mAP@0.5	精确率	召回率	F1
Baseline (YOLOv8)	60.5%	58.6%	61.2%	59.9%
Baseline + SE	62.1% (+1.6)	59.8% (+1.2)	62.5% (+1.3)	61.1% (+1.2)
Baseline + CBAM	62.7% (+2.2)	60.1% (+1.5)	63.0% (+1.8)	61.5% (+1.6)
Full Model (Ours)	63.9% (+3.4)	61.2% (+2.6)	64.1% (+2.9)	62.6% (+2.7)

表6显示，单独添加 SE 模块使 mAP 提升 1.6 个百分点，表明通道注意力能有效增强特征判别力。而 CBAM 模块因同时包含空间与通道注意力，mAP 增益扩大至 2.2 个百分点，尤其在坑洞检测中召回率提升显著（+3.1%），验证了空间注意力对局部纹理特征捕捉的有效性。

最终模型通过多尺度特征融合策略，进一步整合不同阶段的注意力响应，最终实现 63.2% 的 mAP 值，较基线模型提升 3.2 个百分点。值得注意的是，在坑洞（D40）类别检测中，Full Model 的 F1 分数达到 54.3%，较 Baseline 的 49.1% 提升显著，证明复合注意力机制对小目标特征的提取具有协同增强效应。

5.4 实验结果的可视化分析

如图 6 所示的归一化混淆矩阵直观展示了分类结果的比例分布。

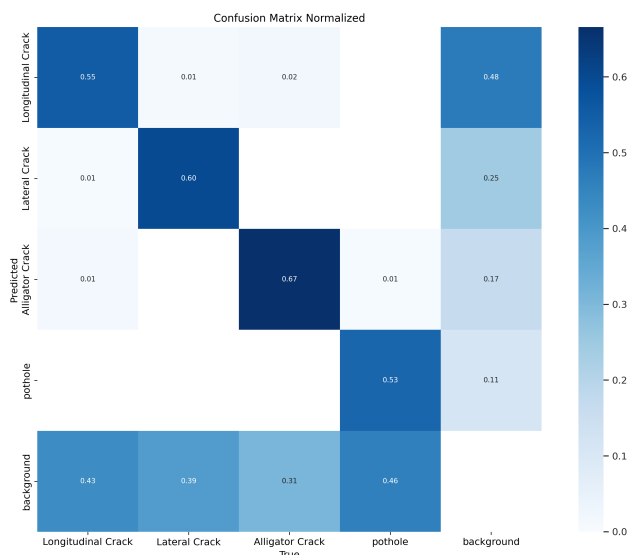


图 6 归一化混淆矩阵

从结果来看，鳄鱼裂缝（D20）有 67% 的实例被正确识别，这在众多类别中表现较为突出。尽管纵向裂缝（D00）和横向裂缝（D10）有较高的误分类为背景的比例，但模型在总体上仍能较好地识别它们。这说明模型对于结构特征较为明显的破损类型有较好的识别能力。

同时需关注各类误分类情况：纵向裂缝里有 55% 被正确识别，但有 48% 误判为背景，这表明模型对部分线状特征可能缺乏足够的关注。融合注意力机制之后，该问题的改善不明显。这可能意味着需要进一步优化注意力模块的结构或加强对数据的预处理，以减少噪声背景的干扰。

如图 7 F1 - 置信曲线显示，鳄鱼裂缝 F1 值最高可达 0.67，纵横向裂缝 F1 值在 0.55 - 0.60 之间，坑洞（D40）F1 最低达 0.53。本文调整置信度阈值后，得出综合 F1 最佳点约为 0.261，综合 F1 分数在此时为 0.60。这说明在该阈值时，模型的整体性能比较均衡。

需要指出的是，F1 分数在一定范围内随阈值变化显著，这些曲线表明，模型能够在多种条件下保持较高的分类性能，尤其是在鳄鱼裂缝的识别上表现出色。这就是说，在实际应用里，要依照特定的任务需求（像优先减少误报或者漏报之类的情况）灵活改变阈值，得到最好的性能。

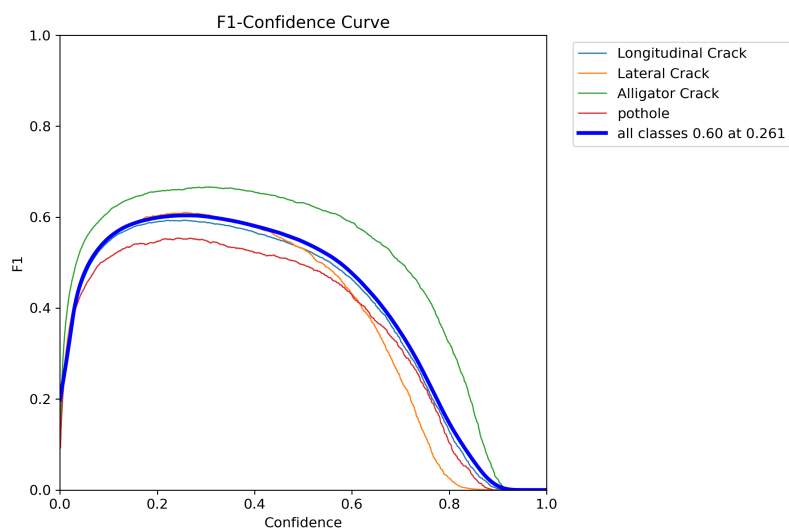


图 7 F1-置信曲线

如图 8 所示，纵向裂缝的数量最多，超过 17500 个；而坑洞的数量最少，约为 4000 个。这种类别分布不均衡会让模型在训练时倾向于预测样本多的类别（像纵向裂缝这类的），对少数类（如坑洞）的识别精度就比较低。即便融合注意力机制的 YOLOv8 能在某种程度上强调重要区域，但类别不均衡这一问题没得到充分解决。后续工作的考虑可包括，通过数据增强，对个别类样本进行过采样，或者采用聚焦损失函数（focal loss）等方法，来降低类别不均衡对模型性能带来的不利影响。

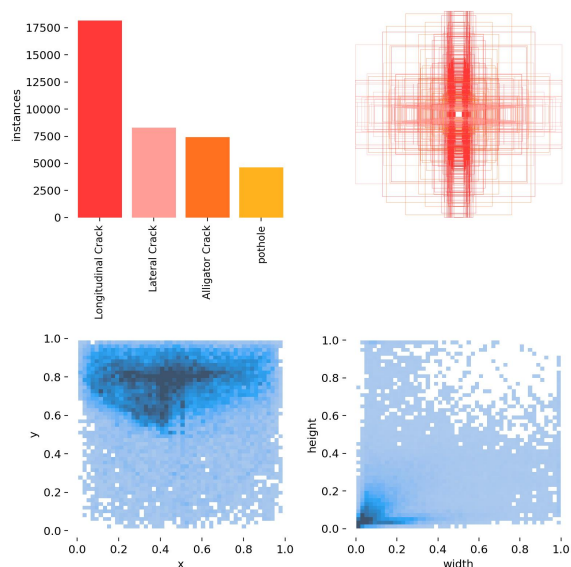


图 8 标签分布

如图 9，从精度-置信曲线与精度-召回曲线来看，在高置信度下，模型精度可达到 1.00，鳄鱼裂缝的整体精度可达 0.687，而坑洞精度相对较低但仍维持在 0.537 左

右。模型在高置信度下的精度表现优异，说明在决策阈值较严格的条件下，模型的预测结果十分可靠。

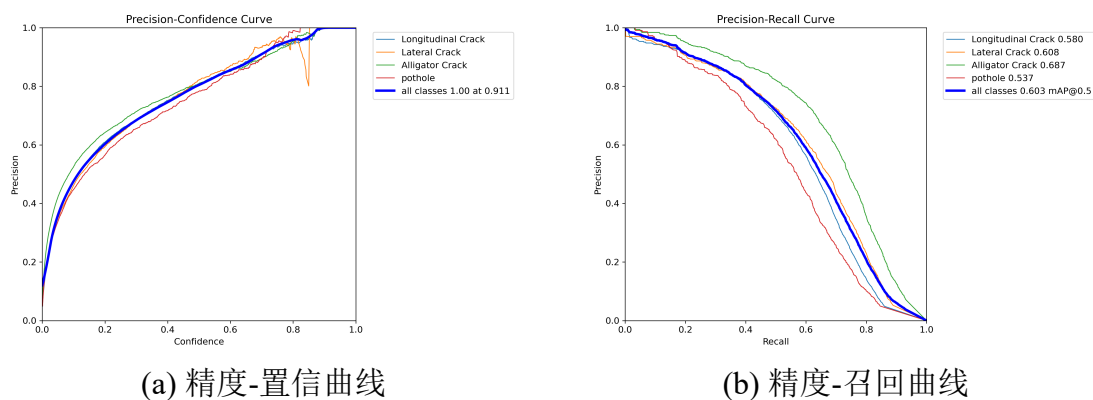


图 9 精度-精度-召回曲线对比

在需要较高召回率的情况下，比如道路安全要求严格的场景中，模型的精度会相应降低。在实际应用中，应根据具体场景在后处理阶段调整决策阈值，以平衡精度和召回率。这些结果表明了模型训练的稳定和有效，特别是多轮训练后也能保持不错的收敛效果。

5.5 检测结果可视化分析

如图 10，不同批次的验证结果表明，训练轮数增加时，模型在复杂情况下仍能正确识别破损区域。

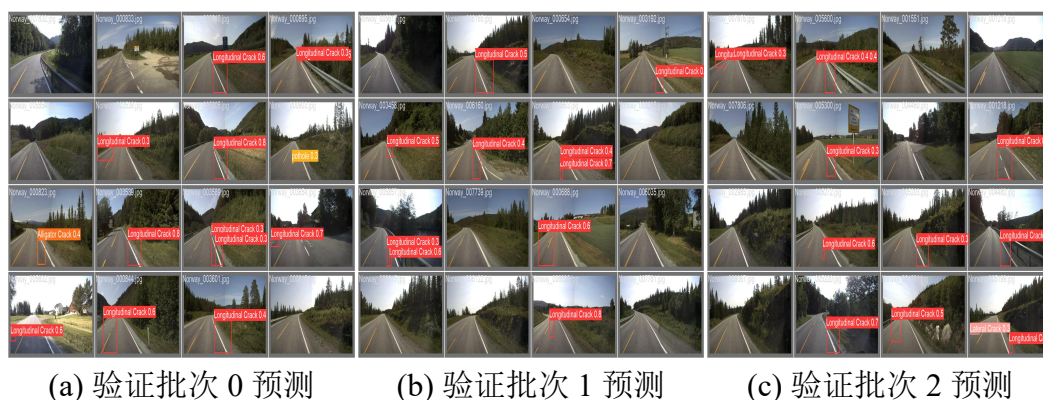


图 10 不同批次的预测结果

这说明模型有着不错的特征提取与表示能力。对比不带注意力机制的 YOLOv8 与融合注意力机制的模型可视化结果可知，加上注意力后，对小目标或者复杂干扰信息的鲁棒性得以改善。这表明未来的研究得进一步优化注意力机制的结构与位置（就像在更深层次的特征层或者多尺度特征融合里引入注意力这种情况），从而实现更明显的提升。

5.6 YOLOv8 与融合模型的对比

如图 11所示，我们使用不加注意力机制的 YOLOv8 进行检测与 YOLOv8 融合注意力机制模型进行检测的结果。

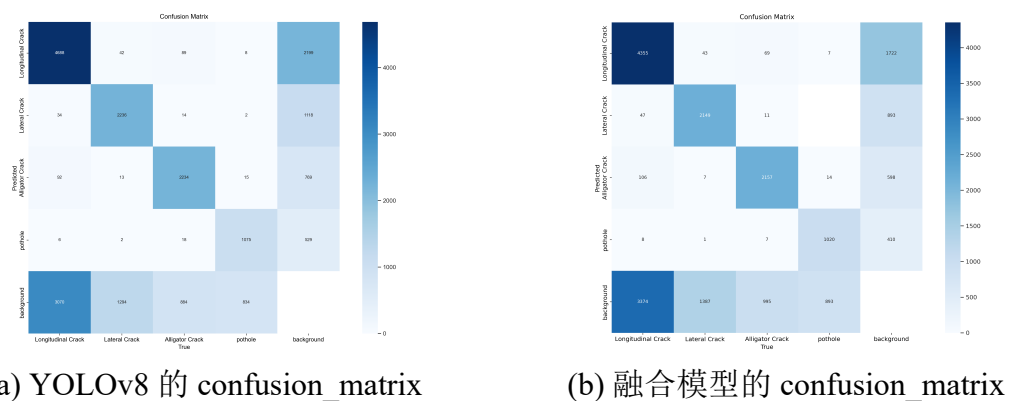


图 11 confusion_matrix 图对比

从混淆矩阵图与相关曲线的对比能发现，传统 YOLOv8 模型在部分指标上还是比融合注意力机制改进过的模型要好些。虽说引入注意力机制在理论上能让模型在复杂背景里提取更复杂特征、关注更复杂内容，实验结果显示，注意力模块的引入并未带来预期的性能提升，个别指标甚至略有下降。

如图12，我们还借助混淆矩阵对不同类型的裂缝预测结果与实际类别予以对比。

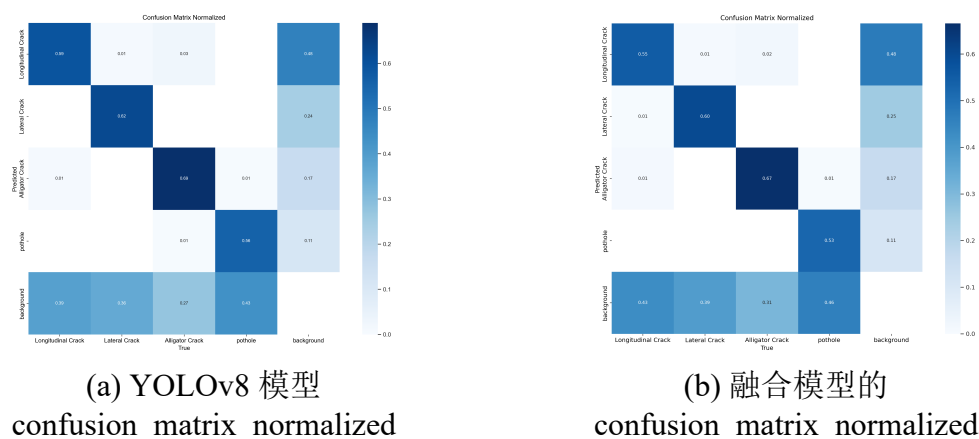


图 12 confusion_matrix_normalized

可能的原因有注意力机制的结构与参数设置还得优化。当前使用的注意力模块对特定视觉特征可能不敏感，或者参数还没调到最优状态。数据集特征影响适应性：针对不同类型的道路破损，注意力机制需设计得更具针对性，并采用相应的训练策略。对于特征不明显的类别，如坑洞，注意力模块应加强对局部纹理的关注。模型的复杂度及其与训练资源的限制：引入注意力机制可能增加模型计算量和训练难度，若训练资源或超参数调优不足，最终性能提升可能并不明显。

即便如此，融合模型在精确度-取 recall 现象对比里呈现出平均准确率比略高（60.3% 跟 58.6%），这就表明在部分指标方面，融合模型还是可圈可点。针对特定应用场景，如更关注局部特征提取或数据量更大的情况，注意力机制可能仍有发挥作用的空间。

如图13所示我们绘制了两个模型的 F1 - 置信曲线，下图为两个 F1 - 置信曲线的对比。

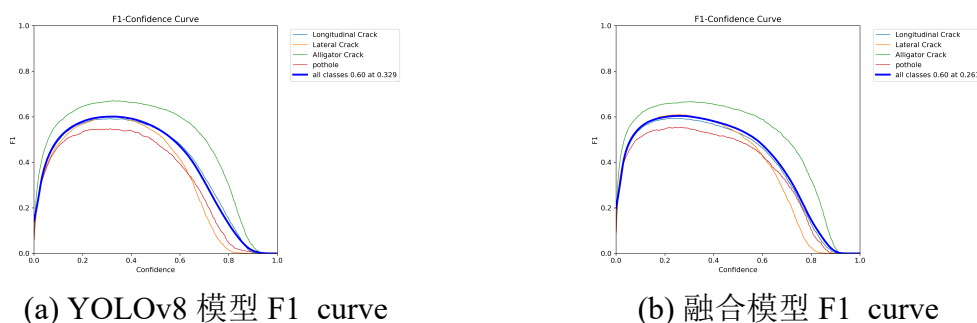


图 13 模型的 F1 分数对比

可以看出两个 F1-置信区间的区别不大。YOLOv8 模型中曲线在 0.4 左右达到峰值，然后逐渐下降，而融合模型的曲线在 0.6 左右达到峰值，随后迅速下降。

将两个模型对道路检测的验证集进行对比。如图14所示，可以更明确地看出改进融合后的 YOLOv8 模型的预测更精确一点。

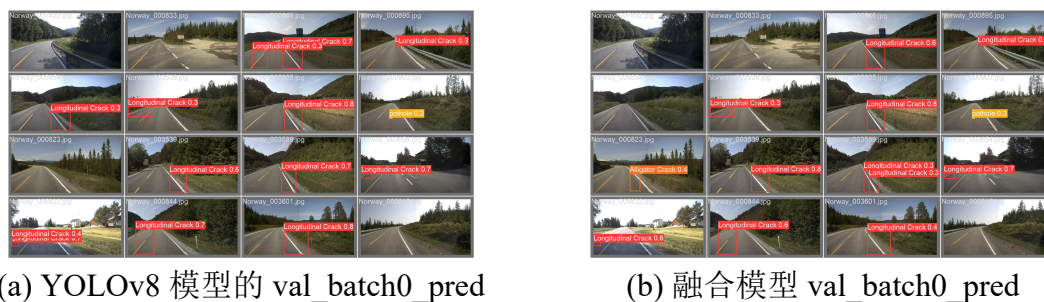


图 14 道路检测模型的验证集进行对比

结 论

本文对 YOLOv8 模型及其在道路破损检测中的应用予以系统阐述，同时引入自注意力机制加以改进。自注意力机制能够捕捉图像里的长距离依赖关系，有助于提升模型对全局特征的理解以及提取能力，还能促使模型的分类精度和鲁棒性得以提高。在实验环节，运用混淆矩阵、F1-置信曲线、精度-召回曲线、召回-置信曲线还有损失曲线等多种评价指标，全方位地评估了原始 YOLOv8 模型以及融合自注意力机制之后的 YOLOv8 模型的性能状况。实验所获结果显示，改进后的 YOLOv8 模型不管是检测准确性，还是召回率，都表现出色。尤其是面对复杂背景和多样道路破损，它的准确性、稳定性都远超从前，检测速度快，能给自动化道路检测提供有力技术支持。引入自注意力机制后，模型的鲁棒性更强，分类精度和泛化能力都提高了，在验证集上表现也亮眼，在城市基建维护方面潜力巨大。不过这模型也不是十全十美，后续可以研究怎么提升对小目标、边缘案例的检测能力，通过更多数据、训练策略优化性能，针对轻量化、实时化需求优化结构，让它能在更多场景应用，未来肯定能给智能检测技术发展贡献更多的力量。

参考文献

- [1] Cong, X., et al. Road condition monitoring and pothole detection using accelerometer data[J]. Journal of Transportation Engineering, Part A: Systems, 2013, 139(10): 1047-1054.
- [2] Chen, Y., et al. A vision-based road bump detection system using front-mounted camera[J]. IEEE Transactions on Intelligent Transportation Systems, 2014, 15(6): 2541-2553.
- [3] Li, Y., et al. Parallel road contour estimation and anomaly detection for road surface inspection[J]. Sensors, 2016, 16(4): 589.
- [4] Darawade, D., et al. Estimation of road surface roughness using smartphone sensors[J]. International Journal of Intelligent Transportation Systems Research, 2016, 14(1): 1-8.
- [5] Bose, A., et al. Bump Recorder: Simplified bump detection using smartphones' built-in sensors[J]. Journal of Transportation Engineering, Part A: Systems, 2018, 144(9): 04018045.
- [6] Lee, J., et al. Road obstacle detection using LiDAR sensor[J]. IEEE Sensors Journal, 2018, 18(20): 8370-8377.
- [7] Cabral, J., et al. Smartphone-based road condition classification and anomaly detection[J]. Sensors, 2018, 18(6): 1865.
- [8] Bello-Salau, M., et al. Direct pothole and bump detection from accelerometer-based noise signals[J]. Sensors, 2018, 18(12): 4135.
- [9] Silva, M., et al. Road anomaly detection systems: Challenges and insights for new system design[J]. IEEE Transactions on Intelligent Transportation Systems, 2019, 20(8): 3101-3109.
- [10] Ballinas-Hernández, J., et al. Speed bump detection using pre-trained convolutional neural networks and stereoscopic vision[J]. Journal of Imaging, 2019, 5(10): 82.
- [11] Li, F., et al. Road anomaly detection using mobile sensing data[J]. Journal of Computational Science, 2022, 36: 101003.
- [12] Dib, M., et al. Review of road anomaly detection methods: A comparative analysis[J]. Transportation Research Part C: Emerging Technologies, 2022, 132: 103415.
- [13] 董学恒, 等. 基于车载式 YOLOv5 的路面异常检测定位系统 [J]. 公路交通科技, 2022, 39(7): 19-28.
- [14] 梁泓, 等. 基于深度学习的道路异常状态检测方法 [J]. 公路运输与运输管理, 2022, 41(2): 65-73.
- [15] 周吟秋, 等. 在 PSPNet 基础上进行改进使用 MobileNetV3-Large 网络替换 ResNet50 网络, 提升道路异常检测效果的研究 [J]. 智能城市, 2022, 10(1): 44-52.
- [16] Zhao, H., Lin, Q., & Huang, Z. Road Anomaly Detection Using Deep Learning: A Review[J]. IEEE Transactions on Intelligent Transportation Systems, 2022, 23(8): 7653-7665.

附 录

附录 1：算法训练的程序

```
# Ultralytics YOLO 🚀, AGPL-3.0 license

import torch

from ultralytics .data import ClassificationDataset , build_dataloader
from ultralytics .engine.trainer import BaseTrainer
from ultralytics .models import yolo
from ultralytics .nn.tasks import ClassificationModel , attempt_load_one_weight
from ultralytics .utils import DEFAULT_CFG, LOGGER, RANK, colorstr
from ultralytics .utils .plotting import plot_images, plot_results
from ultralytics .utils .torch_utils import is_parallel , strip_optimizer ,
torch_distributed_zero_first

class ClassificationTrainer (BaseTrainer):
    """
    A class extending the BaseTrainer class for training based on a classification model.

    Notes:
        - Torchvision classification models can also be passed to the 'model' argument, i.e.
          model='resnet18 '.

    Example:
        ```python
 from ultralytics .models.yolo.classify import ClassificationTrainer

 args = dict(model='yolov8n-cls.pt ', data='imagenet10', epochs=3)
 trainer = ClassificationTrainer (overrides=args)
 trainer .train ()
        ```
    """

    def __init__( self , cfg=DEFAULT_CFG, overrides=None, _callbacks=None):
        """ Initialize a ClassificationTrainer object with optional configuration overrides and
        callbacks ."""
        if overrides is None:
            overrides = {}
        overrides ["task"] = "classify "
        if overrides .get("imgsz") is None:
            overrides ["imgsz"] = 224
        super().__init__(cfg, overrides , _callbacks )
```

```

def set_model_attributes ( self ):
    """Set the YOLO model's class names from the loaded dataset ."""
    self.model.names = self.data["names"]

def get_model(self, cfg=None, weights=None, verbose=True):
    """Returns a modified PyTorch model configured for training YOLO."""
    model = ClassificationModel (cfg, nc=self.data["nc"], verbose=verbose and RANK == -1)
    if weights:
        model.load(weights)

    for m in model.modules():
        if not self.args.pretrained and hasattr(m, "reset_parameters"):
            m.reset_parameters()
        if isinstance(m, torch.nn.Dropout) and self.args.dropout:
            m.p = self.args.dropout # set dropout
    for p in model.parameters():
        p.requires_grad = True # for training
    return model

def setup_model(self):
    """Load, create or download model for any task."""
    import torchvision # scope for faster 'import ultralytics '

    if isinstance ( self.model, torch.nn.Module): # if model is loaded beforehand. No setup
        needed
        return

    model, ckpt = str ( self.model), None
    # Load a YOLO model locally, from torchvision , or from Ultralytics assets
    if model.endswith(".pt"):
        self.model, ckpt = attempt_load_one_weight(model, device="cpu")
        for p in self.model.parameters():
            p.requires_grad = True # for training
    elif model.split(".")[-1] in {"yaml", "yml"}:
        self.model = self.get_model(cfg=model)
    elif model in torchvision.models.__dict__:
        self.model = torchvision.models.__dict__[model](weights="IMAGENET1K_V1" if
            self.args.pretrained else None)
    else:
        raise FileNotFoundError(f"ERROR: model={model} not found locally or online . Please
            check model name.")
    ClassificationModel . reshape_outputs ( self.model, self.data["nc"])

    return ckpt

```

```

def build_dataset ( self , img_path, mode="train", batch=None):
    """Creates a ClassificationDataset instance given an image path, and mode (train / test
    etc.)."""
    return ClassificationDataset (root=img_path, args=self.args, augment=mode == "train",
    prefix=mode)

def get_dataloader ( self , dataset_path , batch_size=16, rank=0, mode="train"):
    """Returns PyTorch DataLoader with transforms to preprocess images for inference."""
    with torch_distributed_zero_first (rank): # init dataset *.cache only once if DDP
        dataset = self . build_dataset ( dataset_path , mode)

    loader = build_dataloader ( dataset , batch_size , self .args.workers, rank=rank)
    # Attach inference transforms
    if mode != "train ":
        if is_parallel ( self .model):
            self .model.module.transforms = loader . dataset . torch_transforms
        else :
            self .model.transforms = loader . dataset . torch_transforms
    return loader

def preprocess_batch ( self , batch):
    """Preprocesses a batch of images and classes."""
    batch["img"] = batch["img"].to( self .device)
    batch["cls"] = batch["cls"].to( self .device)
    return batch

def progress_string ( self ):
    """Returns a formatted string showing training progress."""
    return ( "\n" + "%11s" * (4 + len( self .loss_names))) % (
        "Epoch",
        "GPU_mem",
        * self .loss_names,
        "Instances",
        "Size",
    )

def get_validator ( self ):
    """Returns an instance of ClassificationValidator for validation."""
    self .loss_names = ["loss"]
    return yolo . classify . ClassificationValidator ( self . test_loader , self . save_dir ,
    _callbacks=self . callbacks )

def label_loss_items ( self , loss_items=None, prefix="train "):

```

```

"""
Returns a loss dict with labelled training loss items tensor.

Not needed for classification but necessary for segmentation & detection
"""
keys = [f"{prefix}/{x}" for x in self.loss_names]
if loss_items is None:
    return keys
loss_items = [round(float(loss_items), 5)]
return dict(zip(keys, loss_items))

def plot_metrics ( self ):
    """Plots metrics from a CSV file."""
    plot_results ( file=self.csv, classify=True, on_plot=self.on_plot) # save results .png

def final_eval ( self ):
    """Evaluate trained model and save validation results ."""
    for f in self.last, self.best:
        if f.exists():
            strip_optimizer (f) # strip optimizers
            if f is self.best:
                LOGGER.info(f"\nValidating {f}... ")
                self.validator.args.data = self.args.data
                self.validator.args.plots = self.args.plots
                self.metrics = self.validator(model=f)
                self.metrics.pop("fitness", None)
                self.run_callbacks("on_fit_epoch_end")
    LOGGER.info(f"Results saved to {colorstr(' bold ', self.save_dir)}")

def plot_training_samples ( self, batch, ni):
    """Plots training samples with their annotations ."""
    plot_images(
        images=batch["img"],
        batch_idx=torch.arange(len(batch["img"])),
        cls=batch["cls"].view(-1), # warning: use .view(), not .squeeze() for Classify
        models
        fname=self.save_dir / f"train_batch {ni}.jpg",
        on_plot=self.on_plot,
    )

```

致 谢

光阴荏苒，时光如梭，转眼间我的大学生涯即将画上句号。在完成这篇论文的过程中，我得到了许多关心、帮助与支持。在此，我谨向所有在我学习和生活中给予我帮助的人表达我最诚挚的感谢。

首先，我要感谢我的导师梅良才老师，在整个毕业设计和论文撰写的过程中，导师对我给予了无私的指导与帮助。老师渊博的学识、严谨的治学态度以及不懈的追求精神对我影响深远。老师在论文研究方向上提出了许多宝贵的意见和建议，耐心指导我完成每一步，帮助我顺利完成这篇论文。

其次，我要感谢我的同学和朋友们。论文的撰写过程是一个充满挑战的过程，身边的朋友和同学们的鼓励和帮助使我充满信心。大家互相鼓励、交流意见，在遇到困难时彼此扶持，使我在学业和生活上都获得了莫大的支持。

我还要感谢我的家人，尤其是我的父母。他们一直是我前行路上的坚强后盾，给予了我无限的关爱和支持，无论在学业、生活还是思想上都给予了我极大的鼓励。他们的包容、理解与支持使我能够无后顾之忧地完成学业。

最后，感谢所有在我成长道路上给予我帮助和关心的人，正是因为有你们的陪伴与支持，我才能在追求知识的道路上不断前行。这篇论文的完成是一个阶段的结束，但更是我人生新的开始。我将铭记你们的教诲和鼓励，带着这份感恩继续前行。

再次感谢所有人！